

SMART CITY WASTE COLLECTION ROUTING

Graph Optimization – Technical Report

1. Introduction

Smart-city waste collection is a routing problem in which collection vehicles must visit waste-generation points efficiently while satisfying operational constraints. The road network can be represented as a weighted graph: intersections are vertices, roads are edges, and edge weights represent travel distance, time, fuel cost, or a combined cost. Graph algorithms can then identify efficient paths and construct collection tours.

2. Problem Statement

The objective is to minimize total travel cost and collection time while ensuring that every required collection location is served. A practical system must consider road distance, estimated travel time, traffic congestion, vehicle capacity, depot location, service time, and route feasibility.

3. Graph Model

Let $G = (V, E)$ be the road graph. Each vertex v in V represents an intersection, depot, or waste collection point. Each edge (u, v) in E represents a road segment. A weight $w(u, v)$ can combine distance and traffic-adjusted travel time. For example, a dynamic cost can be defined as $w = \alpha(\text{distance}) + \beta(\text{travel time}) + \gamma(\text{fuel estimate})$, where α , β , and γ are policy-selected weights.

4. Algorithmic Approach

The solution uses Dijkstra's shortest-path algorithm to compute the minimum-cost path from the depot to each collection point. After shortest-path costs are known, a simple nearest-neighbor heuristic builds a complete collection tour by repeatedly selecting the nearest unvisited location that still fits within the vehicle's remaining capacity. When capacity is insufficient, the vehicle returns to the depot, unloads, and starts another trip.

For a production-scale system, the same framework can be extended with A^* , contraction hierarchies, vehicle-routing heuristics, time windows, multiple vehicles, real-time traffic updates, and route re-optimization.

5. Important Constraints

- Traffic conditions: congested roads receive higher travel-time costs, allowing routes to change as traffic changes.
- Vehicle capacity: the vehicle cannot collect more waste than its available capacity.
- Road restrictions: one-way roads, closed roads, weight limits, and restricted zones can be represented by directed or disabled edges.
- Service time: time spent loading waste should be included when estimating route duration.
- Multiple vehicles: vehicles may have different capacities, starting depots, or operating schedules.
- Time windows: some collection locations may only be serviceable during specified periods.

6. Java Implementation

```
import java.util.*;

public class SmartWasteRouting {

    static class Edge {
        int to;
        double distance;
        double trafficTime;

        Edge(int to, double distance, double trafficTime) {
            this.to = to;
            this.distance = distance;
            this.trafficTime = trafficTime;
        }

        double cost() {
            // Combined cost: distance + traffic-adjusted time
            return distance + 0.5 * trafficTime;
        }
    }

    static class Node implements Comparable<Node> {
        int vertex;
        double distance;

        Node(int vertex, double distance) {
            this.vertex = vertex;
            this.distance = distance;
        }

        public int compareTo(Node other) {
            return Double.compare(this.distance, other.distance);
        }
    }

    static class Stop {
        int vertex;
        double waste;

        Stop(int vertex, double waste) {
            this.vertex = vertex;
            this.waste = waste;
        }
    }

    static final double INF = Double.POSITIVE_INFINITY;

    static void addEdge(List<Edge>[] graph, int u, int v,
                        double distance, double trafficTime) {
        graph[u].add(new Edge(v, distance, trafficTime));
        graph[v].add(new Edge(u, distance, trafficTime));
    }

    // Dijkstra: shortest cost from source to every vertex
    static double[] dijkstra(List<Edge>[] graph, int source) {
        int n = graph.length;
        double[] dist = new double[n];
        Arrays.fill(dist, INF);
    }
}
```

```

    dist[source] = 0;

    PriorityQueue<Node> pq = new PriorityQueue<>();
    pq.add(new Node(source, 0));

    while (!pq.isEmpty()) {
        Node cur = pq.poll();

        if (cur.distance > dist[cur.vertex])
            continue;

        for (Edge e : graph[cur.vertex]) {
            double newDist = cur.distance + e.cost();

            if (newDist < dist[e.to]) {
                dist[e.to] = newDist;
                pq.add(new Node(e.to, newDist));
            }
        }
    }

    return dist;
}

// Nearest-neighbor capacity-aware routing heuristic
static void buildRoutes(List<Edge>[] graph, int depot,
                        List<Stop> stops, double capacity) {

    int current = depot;
    double remaining = capacity;
    double totalCost = 0;

    Set<Integer> visited = new HashSet<>();

    System.out.println("Depot -> " + depot);

    while (visited.size() < stops.size()) {

        Stop best = null;
        double bestCost = INF;

        for (Stop stop : stops) {
            if (visited.contains(stop.vertex))
                continue;

            // Skip a stop that cannot fit in the current load.
            if (stop.waste > remaining)
                continue;

            double[] dist = dijkstra(graph, current);

            if (dist[stop.vertex] < bestCost) {
                bestCost = dist[stop.vertex];
                best = stop;
            }
        }

        // No feasible stop: return to depot and unload.
    }
}

```

```

        if (best == null) {
            double[] back = dijkstra(graph, current);
            totalCost += back[depot];

            System.out.println(" -> Depot (unload)");
            current = depot;
            remaining = capacity;
            continue;
        }

        double[] dist = dijkstra(graph, current);
        totalCost += dist[best.vertex];

        remaining -= best.waste;
        current = best.vertex;
        visited.add(best.vertex);

        System.out.println(" -> " + best.vertex +
            " [waste=" + best.waste +
            ", remaining=" + remaining + "]");
    }

    if (current != depot) {
        double[] back = dijkstra(graph, current);
        totalCost += back[depot];
        System.out.println(" -> Depot");
    }

    System.out.printf("Total route cost = %.2f%n", totalCost);
}

public static void main(String[] args) {

    int n = 7;
    List<Edge>[] graph = new ArrayList[n];

    for (int i = 0; i < n; i++)
        graph[i] = new ArrayList<>();

    // distance, traffic-adjusted travel time
    addEdge(graph, 0, 1, 4, 3);
    addEdge(graph, 0, 2, 2, 2);
    addEdge(graph, 1, 3, 5, 5);
    addEdge(graph, 2, 3, 1, 2);
    addEdge(graph, 2, 4, 7, 3);
    addEdge(graph, 3, 5, 3, 2);
    addEdge(graph, 4, 5, 2, 4);
    addEdge(graph, 5, 6, 4, 3);
    addEdge(graph, 1, 6, 10, 8);

    // Waste collection demand at each required location
    List<Stop> stops = Arrays.asList(
        new Stop(1, 3),
        new Stop(3, 4),
        new Stop(4, 2),
        new Stop(5, 3),
        new Stop(6, 2)
    );

    double vehicleCapacity = 7;

```

```
        buildRoutes(graph, 0, stops, vehicleCapacity);  
    }  
}
```

7. Computational Efficiency

For a graph with V vertices and E edges, Dijkstra's algorithm with a binary heap has time complexity $O((V + E) \log V)$. In the demonstration implementation, Dijkstra is called repeatedly by the routing heuristic, so the practical cost is higher than a single shortest-path calculation. For large cities, shortest-path results should be cached or computed once per relevant source, and specialized road-network algorithms can reduce query time substantially.

The route-construction heuristic is fast and easy to implement, but it is not guaranteed to produce the globally optimal vehicle-routing solution. Exact vehicle-routing formulations become computationally difficult as the number of collection points grows, making heuristics and metaheuristics attractive for large urban deployments.

8. Scalability

A large urban environment may contain thousands of intersections and collection locations. The graph should therefore be stored using adjacency lists rather than an adjacency matrix. Real-time traffic data can update edge weights, while route requests can be processed in parallel for different vehicle regions. The city can also be partitioned into zones and solved independently before coordinating boundary routes.

A scalable architecture can use a central road graph, a traffic-data service, a fleet-management module, and a route optimizer. Vehicles periodically report their position, remaining capacity, and collection progress. The optimizer can then recalculate only the affected portions of routes.

9. Interpretation of Results

The algorithm produces a sequence of collection points and depot returns while respecting vehicle capacity. A lower total route cost generally means less travel distance or time, which can reduce fuel use, labor hours, and operating expenses. Avoiding heavily congested roads can further reduce idling and travel time.

From an environmental perspective, shorter and less congested routes can reduce fuel consumption and vehicle emissions. The impact should be measured using operational data such as kilometers driven, fuel consumed, idle time, number of trips, collection completion time, and estimated CO2 emissions.

10. Conclusion

Graph optimization provides a practical foundation for smart-city waste collection. Dijkstra's algorithm is effective for shortest-path computation, while capacity-aware heuristics can convert those paths into usable collection tours. Although a simple heuristic is not guaranteed to be globally optimal, it is computationally practical and can be extended with real-time traffic, multiple vehicles, time windows, and advanced vehicle-routing algorithms. A well-designed system can therefore improve service reliability while reducing fuel cost, travel time, and environmental impact.

11. Expected Benefits

- Reduced total travel distance and fuel consumption.
- Lower collection time and improved fleet utilization.

- Capacity-feasible routes with fewer unnecessary depot trips.
- Better response to traffic changes through dynamic edge weights.
- Scalable routing architecture for large urban road networks.
- Potential reduction in vehicle emissions and operating cost.